IFMT - Instituto Federal de Mato Grosso

Eng. da Computação



Criptografia Assimétrica:

A Evolução da Criptografia Assimétrica

Do princípio matemático de Diffie-Hellman à ameaça da computação quântica

Prof. Dr. Ruy de Oliveira

ROTEIRO DA AULA

Conteúdo

01 O problema fundamental: como compartilhar segredos?

02 Criptografia simétrica e suas limitações

03 O nascimento da criptografia assimétrica

04 Os mecanismos matemáticos: Diffie-Hellman e RSA

05 Curvas elípticas (ECC)

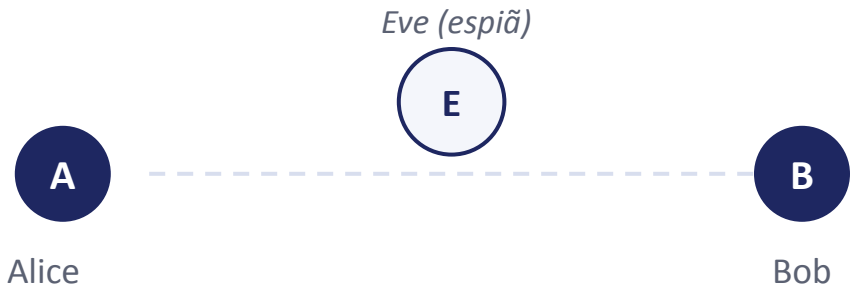
06 A ameaça quântica e o algoritmo de Shor

07 Criptografia pós-quântica

08 Conclusões e o caminho à frente

O dilema de Alice e Bob

- Alice quer enviar uma mensagem secreta para Bob.
- Eve, uma espiã, pode interceptar tudo o que passa pelo canal.
- Pergunta central: como Alice e Bob combinam uma chave secreta sem nunca terem se encontrado antes?



Esse é o problema que atormentou criptógrafos por séculos.

A criptografia simétrica resolve a cifragem — mas cria um novo desafio: o "problema da galinha e do ovo" da distribuição de chaves.

PONTO DE PARTIDA

Criptografia simétrica: rápida, mas com um gargalo

Como funciona

Mesma chave usada para cifrar e decifrar a mensagem.

Exemplos

AES, DES, ChaCha20.

Vantagem

Extremamente rápida — ideal para grandes volumes de dados.

Problema

Como compartilhar a chave com segurança antes de comunicar?

Mesma chave dos dois lados



`C = E(K, M)` `M = D(K, C)`

Se Alice e Bob precisam falar com N pessoas, seriam necessárias N^2 chaves distintas.

1976 — O MARCO

O nascimento da criptografia assimétrica

"New Directions in Cryptography"

Whitfield Diffie e Martin Hellman

- Ideia revolucionária: duas partes podem chegar a um segredo comum sem nunca trocá-lo diretamente.
- Base matemática: dificuldade do logaritmo discreto.

Esse artigo mudou o mundo. Pela primeira vez, era possível estabelecer um segredo sobre um canal totalmente público.

É a base de toda a internet segura hoje.

O protocolo passo a passo

Parâmetros públicos: um número primo **p** e um gerador **g** (conhecidos por todos, inclusive Eve).

ALICE

1. Escolhe **a** secreto
2. Envia **A = g□ mod p**
3. Recebe **B** de Bob
4. Calcula **K = B□ mod p**

BOB

1. Escolhe **b** secreto
2. Envia **B = g□ mod p**
3. Recebe **A** de Alice
4. Calcula **K = A□ mod p**

Ambos chegam ao mesmo segredo K — sem nunca tê-lo transmitido diretamente pelo canal.

Por que o protocolo funciona

$$(g^b)^a = (g^a)^b = g^{ab} \bmod p$$

Propriedade de Exponenciação

Garante que Alice e Bob cheguem exatamente ao mesmo valor **K**, mesmo partindo de cálculos e caminhos diferentes.

Exposição Limitada

Eve intercepta apenas `g`, `p`, `A` e `B` trafegando pelo canal público.

Logaritmo Discreto

Para descobrir **K**, um atacante precisa resolver o problema do logaritmo discreto: dado `g mod p`, encontrar `a`.

Segurança Computacional

Para primos grandes (2048+ bits), esse problema é computacionalmente inviável com computadores clássicos atuais.

Em protocolos reais (ex: TLS com DHE): O servidor envia `p`, `g` e `B = g mod p`; o cliente envia `A = g mod p`.

DIFFIE-HELLMAN EM USO

Diffie-Hellman no dias atuais

Diffie-Hellman na prática: ECDHE e DHE

ECDHE — o padrão de fato

A variante de curva elíptica do Diffie-Hellman, ECDHE (Elliptic Curve Diffie-Hellman Ephemeral), é a forma dominante nas comunicações seguras modernas.

É o método de troca de chaves preferido em TLS 1.3, SSH e IPsec, o ECDH substitui o Diffie-Hellman tradicional por ser drasticamente mais rápido e exigir menor largura de banda devido ao tamanho reduzido de suas chaves.

DHE — legado e ambientes específicos

O Diffie-Hellman clássico sobre corpos finitos (DHE) ainda sobrevive em cenários específicos.

É usado como fallback para sistemas antigos que não suportam curvas elípticas e em algumas APIs internas de empresas que ainda rodam TLS 1.2, geralmente restrito a grupos fortes e padronizados como o FFDHE2048.

Trata-se de um exemplo didático. Na prática esses valores são bem maiores!

```
import random

# PASSO 0 — Parâmetros públicos (Alice e Bob combinam antes, ou o protocolo define)
p = 23   # primo pequeno (em produção: primo de 2048+ bits)
g = 5    # gerador de Z_23* (Evita que o espaço de chaves fique reduzido. N a regra prática é usar grupos padronizados pelo IETF (RFCs).).

# PASSO 1 — Alice escolhe seu segredo e calcula sua chave pública
a = random.randint(2, p - 2)      # segredo de Alice
A = pow(g, a, p)                  # A = g^a mod p
print(f"[Alice] Segredo a = {a}")
print(f"[Alice] Envia A = g^a mod p = {A}")

# PASSO 2 — Bob escolhe seu segredo e calcula sua chave pública
b = random.randint(2, p - 2)      # segredo de Bob
B = pow(g, b, p)                  # B = g^b mod p
print(f"[Bob]   Segredo b = {b}")
print(f"[Bob]   Envia B = g^b mod p = {B}")

# PASSO 3 — Troca pública
print("\n--- Canal público ---")
print(f"Eve vê: p={p}, g={g}, A={A}, B={B}")
print(f"Eve NÃO vê: a={a}, b={b}")
```

```
# PASSO 4 — Alice calcula a chave compartilhada
# Alice recebe B e calcula: K_A = B^a mod p = (g^b)^a mod p = g^(ab) mod p

K_alice = pow(B, a, p)
print(f"\n[Alice] Calcula K = B^a mod p = {B}^{a} mod {p} = {K_alice}")

# PASSO 5 — Bob calcula a chave compartilhada
# Bob recebe A e calcula: K_B = A^b mod p = (g^a)^b mod p = g^(ab) mod p
K_bob = pow(A, b, p)
print(f"[Bob]   Calcula K = A^b mod p = {A}^{b} mod {p} = {K_bob}")

# PASSO 6 — Verificação: as chaves devem ser IGUAIS

print(f"\n[Verificação] K_alice == K_bob ? {K_alice == K_bob}")
print(f"[Chave compartilhada] K = {K_alice}")

# A partir daqui, Alice e Bob podem usar K como chave de sessão
# (por exemplo, como chave de um AES) para se comunicarem com segurança.
```

1977 — UM PASSO ALÉM

RSA: criptografia assimétrica completa

- Criado por Ron Rivest, Adi Shamir e Leonard Adleman, no MIT.
- Vai além do Diffie-Hellman: permite cifrar mensagens diretamente e assinar digitalmente.
- Base matemática: dificuldade de fatorar números inteiros muito grandes.

Diffie-Hellman criava apenas uma chave compartilhada. O RSA permitiu cifrar diretamente com uma chave pública — um avanço gigantesco.

Fórmula central

$$C = M^e \bmod n$$

Cifra a mensagem M usando o expoente público e e o módulo n.

PARA APROFUNDAR

Referências

- Diffie, W. & Hellman, M. (1976) — New Directions in Cryptography.
- Rivest, R., Shamir, A. & Adleman, L. (1978) — A Method for Obtaining Digital Signatures and Public-Key Cryptosystems.
- Shor, P. (1994) — Algorithms for Quantum Computation: Discrete Logarithms and Factoring.
- NIST — Post-Quantum Cryptography Standardization (2024).
- Stallings, W. — Cryptography and Network Security.
- Boneh, D. & Shoup, V. — A Graduate Course in Applied Cryptography.